

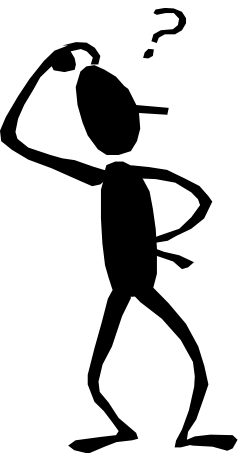
软件构造

-----第3章 软件构建

主讲：彭辉

Email: penghui@cuit.edu.cn

本章目标



1. 了解什么是可以工作的类、什么是高质量的子程序、什么是防御式编程、什么是变量和语句构建；
2. 能够采用编码规范和代码版本控制工具对软件构造阶段的代码进行管理

这章主要内容来自于《代码大全》

产品特点

软件开发奠基之作
近三十年百万现象级科技类畅销书
覆盖常识、习惯和思维的实务指南

代码大全 CODE COMPLETE

2
纪念版

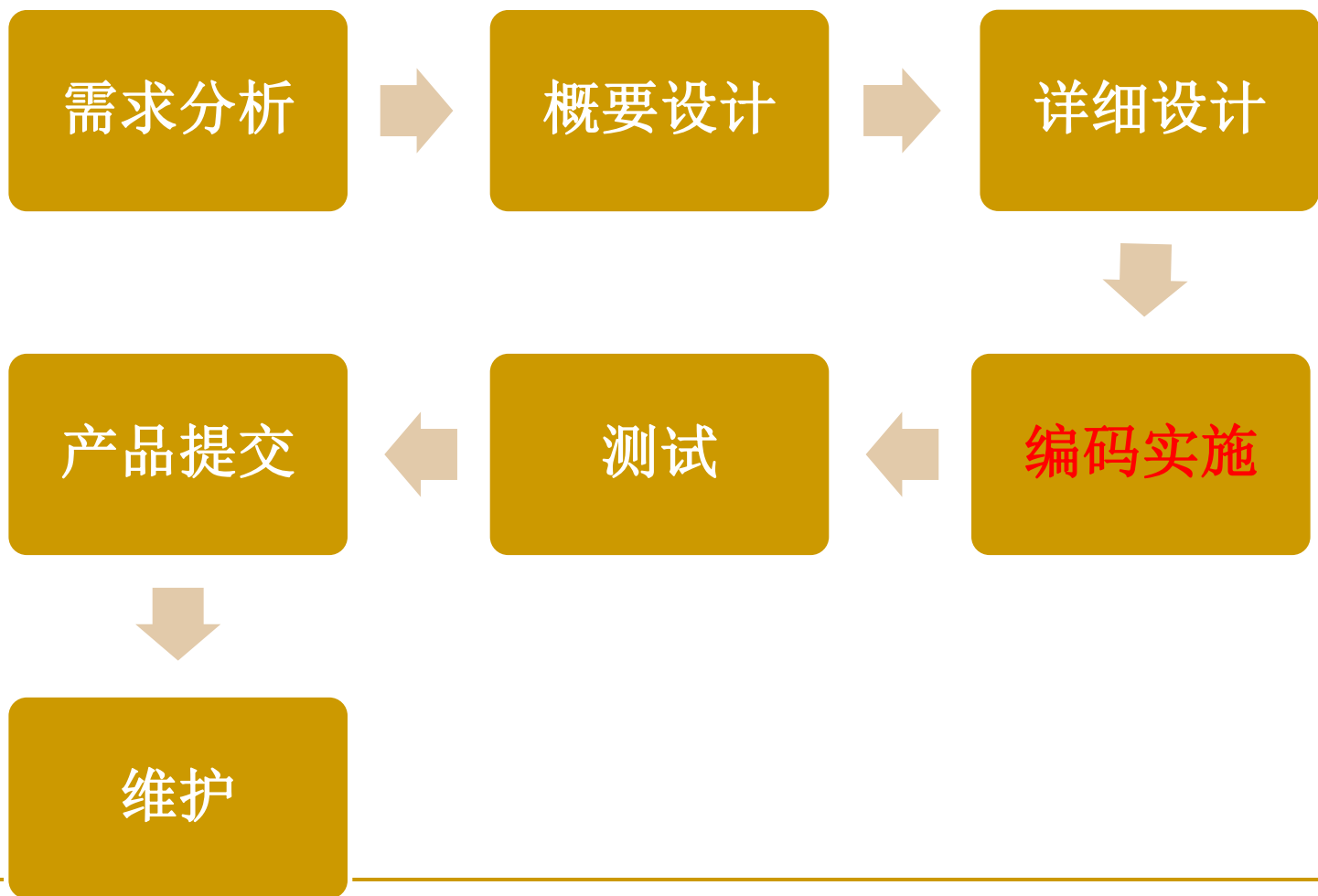


《软件开发》杂志JOLT大奖

第3章 软件构建

- 3.1编码概述
 - 3.2编程语言和编码规范
 - 3.3代码版本控制
 - 3.4可以工作的类
 - 3.5高质量的子程序
 - 3.6防御式编程
 - 3.7变量和语句的构建
-

软件项目开发流程图



类比：项目编码实施



第3章 软件构建

- 3.1编码概述
 - 3.2编程语言和编码规范
 - 3.3可以工作的类
 - 3.4代码版本控制
 - 3.5高质量的子程序
 - 3.6防御式编程
 - 3.7变量和语句的构建
-

3.1 编码概述

1、编程语言发展历史：

- 第一代编程语言：机器语言
- 第二代编程语言：汇编语言
- 第三代编程语言：高级程序设计语言，包括 BASIC、Fortran、C、C++、JAVA、Python 等
- 第4代和第5代编程语言：分类有争议，智能化语言、新兴语言如 Prolog 等

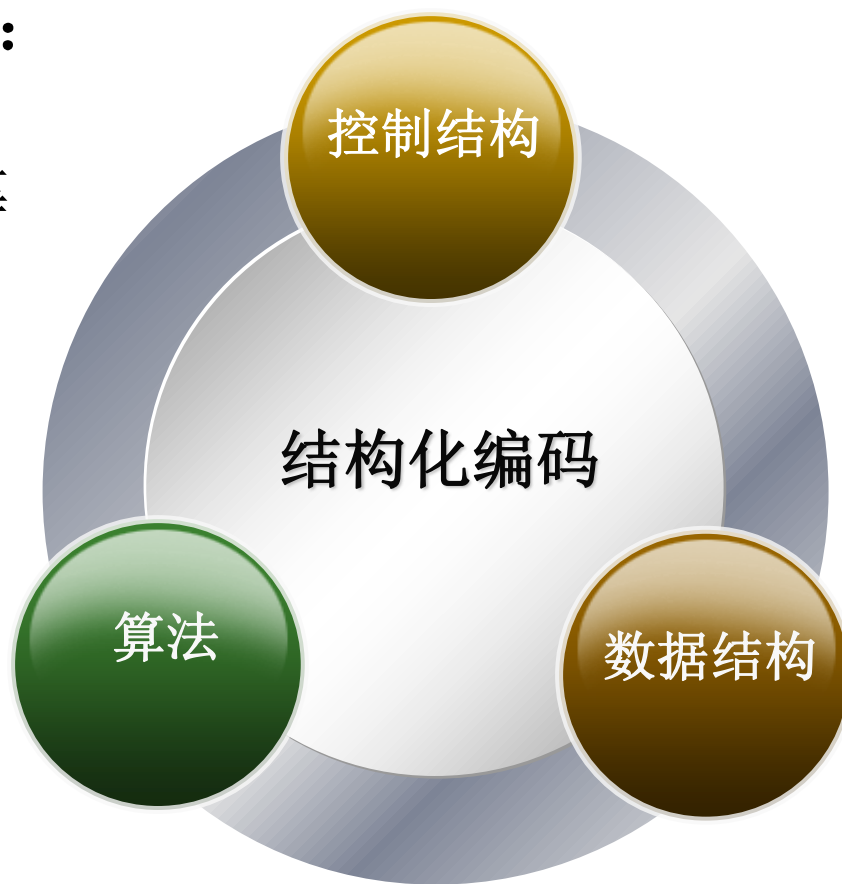
3.1 编码概述

2、编程模式：

- 面向过程（结构化）编程
- 面向对象编程
- 面向组件编程
- 面向服务编程
- 低代码编程模式

面向过程（结构化）编程

面向过程编程语言：
BASIC、C、
Fortran、Pascal等

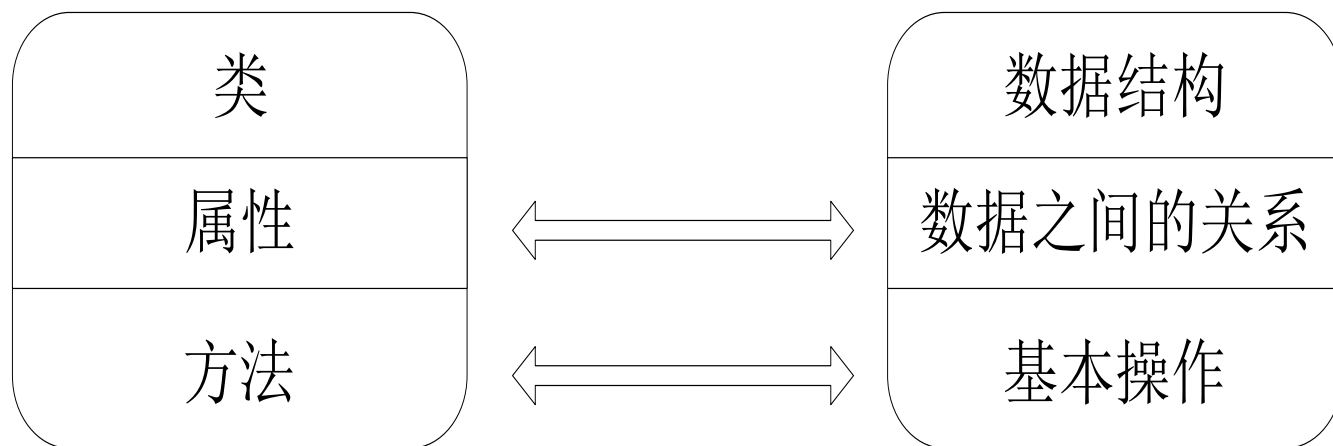


面向过程编程

- 数据结构
- 控制结构
- 数据和操作的分离

面向对象编程

- ❑ 面向对象的编程语言最大的特色就是可以编写自己所需的数据类型
- ❑ 数据结构与面向对象具有天然的对应
- ❑ 面向对象编程语言有：C++、JAVA、Python等



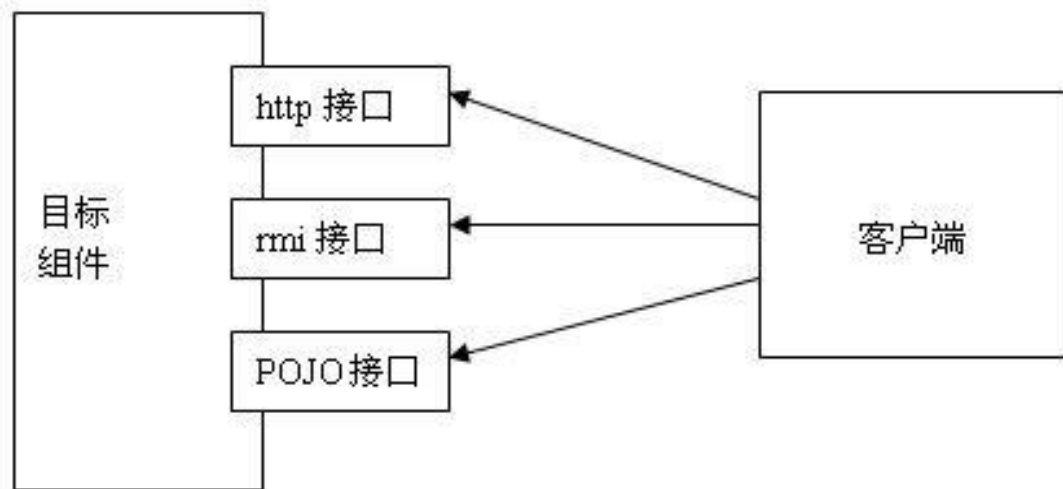
面向组件编程

- 面向组件着眼于独立工作的可替换的程序集或模块，并且无需了解其内部工作原理。
- 面向组件的开发人员大部分时间花在设计接口上，而不是花力气设计复杂的类层次结构。
- 面向组件技术是对面向对象技术的进一步发展，面向对象的设计原则也适用于面向组件。
- 粗粒度的复用
- 一个中间件，甚至一个框架
- C/C++动态链接库DLL是一种组件技术，JAVA中的JavaBean和EJB规范也是一种组件技术

面向组件编程（COP）

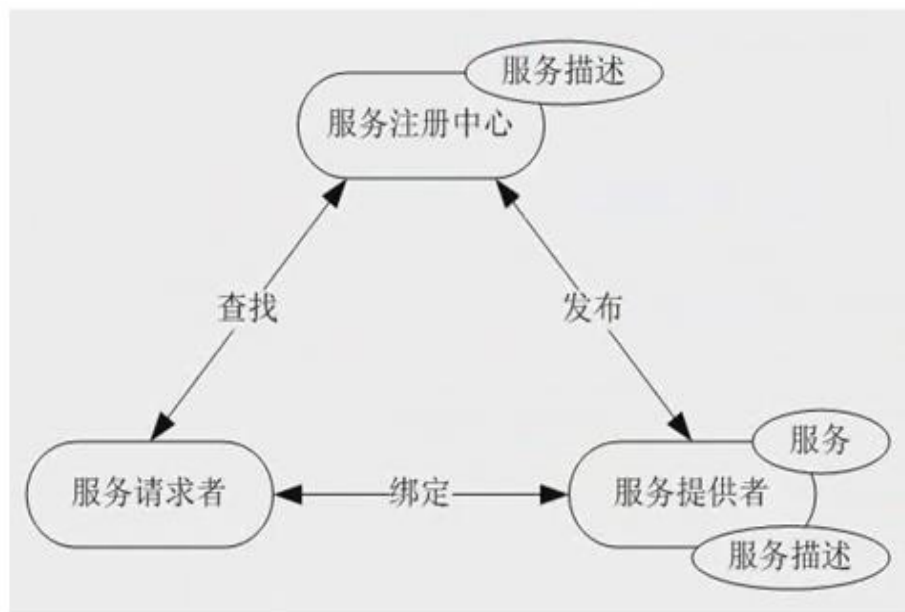
组件（component）：实现了一组服务

Spring框架中就采用了COP的思路，将系统看作一个个的组件，通过定义组件之间的协作关系（通过服务）来完成系统的构建。



面向服务编程（SOA）

面向服务以服务为出发点，组织和协调相关的对象来提供目标服务，对外提供必要的参数输入接口，将服务的结果作为输出，而“服务”本身的计算过程和组织的封装在一起，对用户透明。



面向服务(Service-Oriented)



低代码编程模式

低代码开发平台是一种aPaaS（Application Platform as a Service），它是仅需少量编码甚至无需编码 (0 代码) 即可快速通过可视化拖拽 (drag & drop) 的方式完成应用程序开发的平台。该名词最早于2014年6月由Forrester Research最先提出。

低代码开发平台特点：

- ✓ 可视化集成开发环境（Visual IDE）；
- ✓ 大量可重用且支持拖拽的组件（drag & drop）
- ✓ 等等
- ✓ DeepSeek写代码（人工智能）

第3章 软件构建

- 3.1编码概述
 - 3.2编程语言和编码规范
 - 3.3可以工作的类
 - 3.4代码版本控制
 - 3.5高质量的子程序
 - 3.6防御式编程
 - 3.7变量和语句的构建
-

3.2 编程语言和编码规范

3.2.1 编程语言

- 有多少种编程语言？各编程语言的流行度(所占的市场份额)？
- **TIOBE**组织，对**20**多种语言进行统计排行
网址：
<http://www.tiobe.com/index.php/content/paperinfo/tpci/index.html>

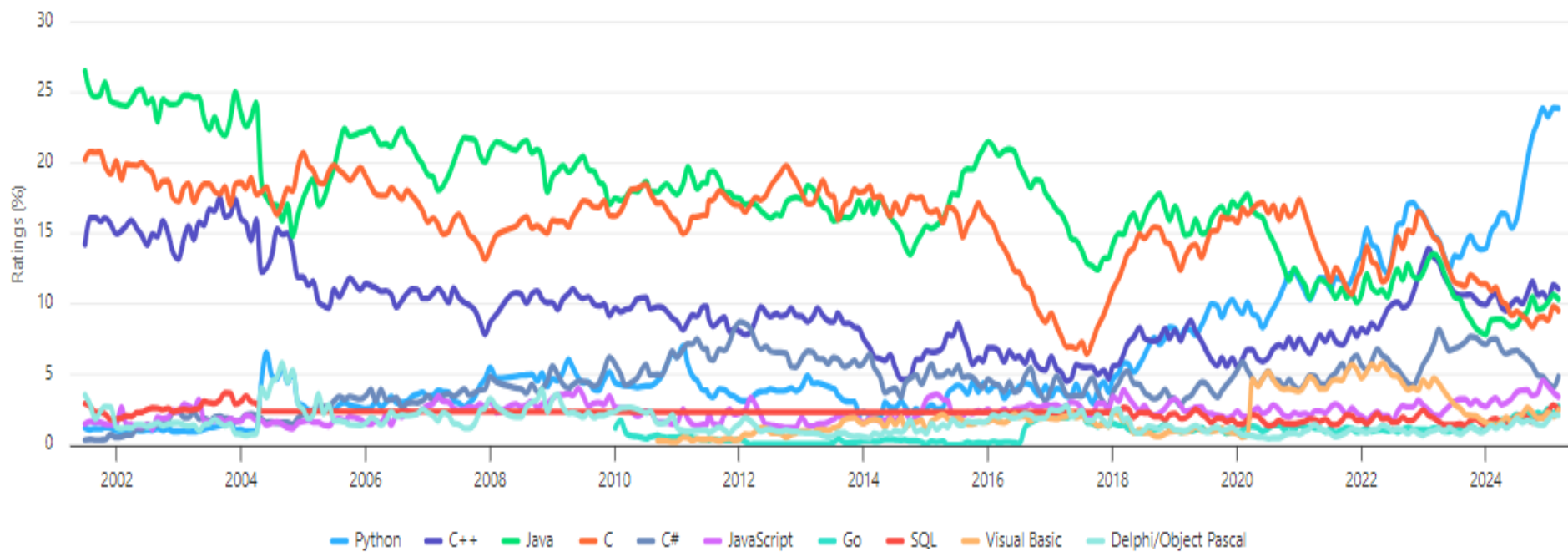
TIOBE 发布的2025年3月份的编程语言排行榜

Mar 2025	Mar 2024	Change	Programming Language		Ratings	Change
1	1			Python	23.85%	+8.22%
2	3	▲		C++	11.08%	+0.37%
3	4	▲		Java	10.36%	+1.41%
4	2	▼		C	9.53%	-1.64%
5	5			C#	4.87%	-2.67%
6	6			JavaScript	3.46%	+0.08%
7	8	▲		Go	2.78%	+1.22%
8	7	▼		SQL	2.57%	+0.65%
9	10	▲		Visual Basic	2.52%	+1.09%
10	15	▲▲		Delphi/Object Pascal	2.15%	+0.94%
11	14	▲		Fortran	1.70%	+0.48%
12	9	▼		Scratch	1.66%	+0.21%
13	12	▼		PHP	1.48%	+0.16%

编程语言走势图

TIOBE Programming Community Index

Source: www.tiobe.com



个人如何选择程序设计语言

- 根据爱好，选择一个**TOP5**的语言
- 至少要精通一门语言(很重要)
- 如何精通一门语言？
知道食谱不一定成为名厨，没有捷径，必须
不断实践

3.2.2 编码规范

- 代码组织与风格
- 注释
- 命名
- 声明
- 语句结构

Java开发手册、编程规范（**Java**）.doc、编码规范（**C++**）.doc，请同学们查阅资料并结合自己编程体会，谈谈怎样按照规范编写代码？

代码组织与风格

1 基本原则

- 代码组织的目的：美观
- 代码的组织 and 风格的基本原则是：便于自己的开发，易于与他人的交流。
- 因个人习惯和编辑器等可以设置和形成自己的风格，但必须前后一致，并符合本规范的基本要求和原则。

2 缩进

- 子功能块当在其父功能块后缩进。
- 当功能块过多而导致缩进过深时将子功能块提取出来做为子函数。
- 代码中以**TAB**（4个字符）缩进，在编辑器中请将**TAB**设置为以空格替代，否则在不同编辑器或设置下会导致**TAB**长度不等而影响整个程序代码的格式。例如：

缩进示例：

```
public void methodName(){  
    if(some condition){  
        for(...){  
            //some sentences  
        }//end for  
    }//end if  
}
```



fun()

3长度

- 为便于阅读和理解，单个函数的有效代码长度当尽量控制在**100**行以内（不包括注释行），当一个功能模块过大时往往造成阅读困难，因此当使用子函数等将相应功能抽取出来，这也有利于提高代码的重用度。
- 单个类也不宜过大，当出现此类情况时当将相应功能的代码重构到其他类中，通过组合等方式来调用，建议单个类的长度包括注释行不超过**1500**行。
- 尽量避免使用大类和长方法。

4行宽

页宽应该设置为**80**字符。一般不要超过这个宽度,这会导致在某些机器中无法以一屏来完整显示,但这一设置也可以灵活调整。在任何情况下,超长的语句应该在一个逗号后或一个操作符前折行。一条语句折行后,应该比原来的语句再缩进一个**TAB**或4个空格,以便于阅读。

```
public Connection getConnection(String url, ↵
                                String userName, ↵
                                String password) ↵
    throws SQLException, IOException{ ↵
} ↵

public final static String SQL_SELECT_PRODUCT = "SELECT * " ↵
+ " FROM TProduct WHERE Prod_ID = " ↵
```

5间隔

- 类、方法及功能块间等应以空行相隔，以增加可读性，但不得有无规则的大片空行。
- 操作符两端应当各空一个字符以增加可读性。
- 相应独立的功能模块之间可使用注释行间隔，并标明相应内容

```
public class Mymodel {  
    public static void main(String[] args) {  
        StudentScore ss=new StudentScore();  
        //调用getScore方法  
        ss.getScore();  
    }  
}  
  
class StudentScore {  
    float[] score = {88, 98, 77, 83, 90, 43, 52}; //从数据库获取....  
    float maxScore; //最高分  
    float aveScore; //平均分  
    float minScore; //最低分  
    .....  
}
```

```
*函数名: getMaxScore  
*功能: 计算最高分  
*返回值: float  
*/  
float getMaxScore() {  
    maxScore=score[0];  
    for(int i=0;i<score.length;i++) {  
        if(maxScore<score[i])  
            maxScore=score[i];  
    }  
    return maxScore;  
}  
  
/**  
*函数名: getMinxScore  
*功能: 计算最低分  
*返回值: float  
*/  
float getMinxScore() {  
    minScore=score[0];  
    for(int i=0;i<score.length;i++) {  
        if(minScore>score[i])  
            minScore=score[i];  
    }  
    return minScore;  
}
```

6 对齐

- 关系密切的行应对齐，对齐包括类型、修饰、名称、参数等各部分对齐。
- 连续赋值时当对齐操作符。
- 当方法参数过多时当在每个参数后（逗号后）换行并对齐。
- 当控制或循环中的条件比较长时当换行（操作符前）、对齐并注释各条件。
- 变量定义最好通过添加空格形成对齐，**同一类型的变量应放在一起**

/*变量对齐-----*/

```
int count          = 100;
int length         = 0;
String strUserName = null;
Integer[] porductCode = new Integer(2); //产品编码数组
/*参数对齐-----*/
public Connection getConnection(String url,
                                String userName,
                                String password)
    throws SQLException,IOException{
}
```

/*换行对齐-----*/

```
public final static String SQL_SELECT_PRODUCT = "SELECT * "
    + " FROM TProduct WHERE Prod_ID = "
    + prodID;
```

/*条件对齐-----*/

```
if( Condition1 //当条件一
    && Condition2 //并且条件二
    || Condition3){ //或者条件三
}
```

/*相应独立的功能模块之间可使用注释行间隔，并标明相应内容*/

```
for(int i = 0;
    i < productCount.length; //循环终止条件
    i++){
}
```

- 以下是两个断开算术表达式的例子。前者更好，因为断开处位于括号表达式的外边，这是个较高级别的断开。

```
longName1 = longName2 * (longName3 + longName4 - longName5)  
            + 4 * longname6;
```

```
longName1 = longName2 * (longName3 + longName4  
                        - longName5) + 4 * longname6;
```

7 括号

■ `{}` 中的语句应该单独作为一行，左括号"`{`"当紧跟其语句后，右括号"`}`"永远单独作为一行且与其匹配行对齐，并尽量在其后说明其匹配的功能模块。

较长的方法以及类、接口等的右括号后应使用 `//end ...` 等标识其结束。如：

类的结束符： `}//EOC ClassName,`

方法结束符： `}//end methodName(),`

功能块结束： `}//end if...userName is null?`

循环快结束： `}//end for...every user in userList`

左括号是否换行等随个人习惯而定，若换行则当与其前导语句首字符对齐。

注释

- 注释分为序言性注释和功能性注释
- 序言性注释应置于每个模块的起始部分，主要内容有：
 - ①说明每个模块的用途、功能。
 - ②说明模块的接口：调用形式、参数描述及从属模块的清单。
 - ③数据描述：重要数据的名称、用途、限制、约束及其他信息。
 - ④开发历史：设计者、审阅者姓名及日期，修改说明及日期。

- 功能性注释嵌在源程序体中，用以描述其后的语句或程序段是在做什么工作，或是执行了下面的语句会怎么样。而不要解释下面怎么做。

□ 例如：

1)不好。 */* ADD AMOUNT TO TOTAL */*

TOTAL = AMOUNT + TOTAL

2)好。注明把月销售额计入年度总额，便使读者理解了下面语句的意图：

/ ADD MONTHLY-SALES TO ANNUAL-TOTAL */*

TOTAL = AMOUNT + TOTAL

1注释基本原则

- (1) 注释应该增加代码的清晰度。代码注释的目的是要使代码更易于被其他开发人员等理解。
- (2) 避免使用装饰性内容。
- (3) 保持注释的简洁。
- (4) 注释信息不仅要包括代码的功能，还应给出原因。
- (5) 不要为注释而注释。
- (6) 除变量定义等较短语句的注释可用行尾注释外，其他注释当避免使用行尾注释。

2文件注释

- 在每个文件的头部都应该包含该文件的功能、作用、作者、版权以及创建、修改记录等。

/*

=====

* **FileName:** helloworld.java

* **Created:** [2006-9-6 20:18:53] by XX

*

=====

* **Copyright:** Copyright (c) 成都信息工程学院, 2015-2021

*

=====

* **Description:** 在这儿对整个文件进行描述

*

=====

• **Modification History:**

• 对这个文件的修改历史进行详细描述，一般包括版本，时间，修改者，修改的内容描述，修改的原因。

例如：（1）V1.0.0.0, 2006-9-6, XX, 创建了此文件

* （2）V1.0.0.1, 2006-10-20, XX, 增加了**功能，张山提出的。

3类、接口注释

- 在类、接口定义之前当对其进行注释，包括类、接口的目的、作用、功能、继承于何种父类，实现的接口、实现的算法、使用方法、示例程序等。

```
/******
```

类名： **StringUtil**

功能： 定义字符串操作时所需要用到的方法，如转换中文、**HTML**标记处理等。

作者： **XX**

版本： **1.0版**

备注：

```
*****/
```

```
public class StringUtil {
```

```
...
```

```
}
```

4 方法注释

- 明确该方法功能、作用、各参数含义以及返回值等。
- 参数注释时当注明其取值范围等。
- 返回值当注释出失败、错误、异常时的返回情况。
- 异常当注释出什么情况、什么时候、什么条件下会引发什么样的异常

例1:

/******

函数名: executeQuery

功能: 该方法根据SQL语句，查询数据库，返回它的结果集

参数: 1、sql[in]:标准的SQL语句，不可以为空串

返回值: ResultSet结果集，若查询失败则返回null

异常: SQLException，当查询数据库时可能引发此异常

备注:

*****/

```
public ResultSet executeQuery(String sql) throws SQLException
{
```

```
    //Statement和SQL语句都不能为空
```

```
    if(null != stmt && !StringUtil.isEmpty(sql)){
```

```
        //返回查询执行结果
```

```
        return stmt.executeQuery(sql);
```

```
    }
```

```
    return null;
```

```
}//end executeQuery()
```

5 其他注释

- 应对重要的变量加以注释，以说明其含义等。
- 应对不易理解的分支条件表达式加注释。不易理解的循环，应说明出口条件。过长的方法实现，应将其语句按实现的功能分段加以概括性说明。
- 对于异常处理，当注明正常情况及异常情况的条件，并说明当异常发生时程序当如何处理。

注释在源代码中的比例一般为**20%左右**

Java注释

■ 行注释

行注释使用“//”进行标记，用于对某行代码进行注释。可尾随在某行代码后，也可以单独成一行。

```
int i = 0;// 定义变量i
```

```
//定义变量j
```

```
int j = 0;
```

Java注释

■ 块注释

块注释使用 “/*.....*/”进行标记，通常用于注释多行代码或用于说明文件、方法、数据结构等的意义与用途。

```
/* main方法负责输入hello  
   与iuiyui  
   */  
public static void main(String[] args) {  
    //代码省略  
}
```

Java注释

■ 文档注释

文档注释使用 “`/**.....*/`”进行标记，并写入javadoc文档。注释文档将用来生成HTML格式的代码报告，所以注释文档必须书写在类、域、构造函数、方法，以及字段（**field**）定义之前。

```
/**
 * <h1>main方法负责输出Hello</h1>
 * @param args
 * @return
 */
public static void main(String[] args) {
    int i = 0; // 定义变量i
    // 其余代码省略
}
```

```
/**
 * .....
 * .....
 */
```

@用来表示版本、作者等

命名

1命名基本原则

规范的命名能使程序更易阅读，从而更易于理解。它们也可以提供一些标识功能方面的信息，有助于更好的理解代码和应用。

- 名字应能反映它所代表的实际东西，应有一定实际意义。例如，表示次数的量用**Times**，表示总量的用**Total**，表示平均值的用**Average**，表示和的量用**Sum**等
- 使用可以准确说明变量/字段/类/接口/包等的完整的英文描述符。例如，采用类似 **firstName**，**listAllUsers** 或 **CorporateCustomer** 这样的名字，尽可能不要使用汉语拼音及不相关单词命名。
- 采用该领域的术语。如果用户称他们的“客户” (**clients**) 为“顾客” (**customers**)，那么就采用术语 **Customer** 来命名这个类，而不用 **Client**。

- 采用大小写混合，提高名字的可读性。一般应该采用小写字母，但是类和接口的名字的首字母，以及任何中间单词的首字母应该大写。
- 尽量少用缩写，但如果一定要使用，当使用公共缩写和习惯缩写等，如实现（**implement**）可缩写成**impl**，经理（**manager**）可缩写成**mgr**等，严禁滥用缩写。
- 避免使用长名字（最好不超过 **25** 个字母）。
- 避免使用相似或者仅在大小写上有区别的名字。

2类、接口

- 所有单词首字母大写。使用能确切反应该类、接口含义、功能等的词。一般采用名词。

例： **class StringList;**

3 常量

- 采用完整的英文大写单词，在词与词之间用下划线连接，如：

Static final int DEFAULT_VALUE = 10;

4 变量和参数

四种变量命名法:

- 匈牙利命名法
- 驼峰式命名法
- 帕斯卡命名法
- 下划线命名法

(1) 匈牙利命名法

是一套由微软的一个匈牙利软件工程师首先使用，并且在微软推广开来，现在被称之为匈牙利命名法的命名规则。匈牙利命名法规定，**变量命名由（前缀+修饰语）构成**。使用表示标识符所对应的变量类型的英文小写缩写作为标识符的前缀，后面在使用表示变量意义的英文单词或缩写进行命名。下面是匈牙利命名法中的一些命名前缀：

类型前缀：用**s(String)**表示字符串，**c(char)**表示字符，**n(number)**数值，**i(intger)**表示整数，**d(double)**表示双精度，**f(float)**浮点型，**b(bool)**布尔型，**dt(date)**表示日期型。

例如：**iLength**表示整型的局域变量，是用来标识长度，**sCode**表示字符串型的局域变量，用来标识代码，**bFlag**，**nCount**

（2）驼峰式命名法

- 驼峰式命名法（**Camel-Case**）是指混合使用大小写字母来构成变量和函数的名字。骆驼式命名法就是当变量名或函数名是由一个或多个单词连结在一起，而构成的唯一识别字时，第一个单词以小写字母开始；从第二个单词开始以后的每个单词的首字母都采用大写字母，例如：**myFirstName**、**myLastName**，这样的变量名看上去就像骆驼峰一样此起彼伏，故得名。

(3) 帕斯卡命名法

- 帕斯卡命名法，又叫大驼峰式命名法。与小驼峰式命名法的最大区别在于，每个单词的第一个字母都要大写。

例子：

```
int MyAge;
```

```
char MyName[10];
```

```
float ManHeight;
```

(4) 下划线命名法

- 下划线命名法并不如大小驼峰式命名法那么备受推崇，但是也是浓墨重彩的一笔。尤其在宏定义和常量中使用比较多，通过下划线来分割全部都是大写的单词。该命名规范，也是很简单，要求单词与单词之间通过下划线连接即可
- 例子：`int my_age;`
- `char my_name[10];`
- `float man_height;`

5 组件/控件

- 应采用完整的英文描述符命名组件（接口部件），遵循匈牙利命名法则 如：**btnOK**，**lblName**。
- 控件命名由（**前缀+修饰语**）构成。前缀即为控件的名称或缩写。

控件种类	前缀	例
命令按钮	btn	btnSave
组合框	cmb	cmbCondition
文本框	txt	txtUserName
标签	lbl	lblOpenDate
列表框	lst	lstSelectedItem
检查框	chk	chkKey

6 方法

方法的命名应采用完整的英文描述符，大小写混合使用：所有中间单词的第一个字母大写。方法名称的**第一个单词常常采用一个有强烈动作色彩的动词**。

取值类使用**get**前缀，设值类使用**set**前缀，判断类使用**is(has)**前缀。

例：

getName()

setSarry()

isLogin()

checkUser()

声明

1 方法

良好的程序设计应该尽可能减小类与类之间耦合，所遵循的经验法则是：尽量限制成员函数的可见性。如果成员函数没必要公有 (**public**)，就定义为保护 (**protected**)；没必要保护 (**protected**)，就定义为私有 (**private**)。

声明顺序：

- ❑ 构造方法
- ❑ 静态公共方法
- ❑ 静态私有方法
- ❑ 公共方法
- ❑ 受保护方法
- ❑ 私有方法

2 字段（成员变量）

- 若没有足够理由，不要把实例变量或类变量声明为公有。公共和保护的可可见性应当尽量避免，所有的字段都建议置为私有，由获取和设置成员函数（**getX**、**setX**）访问。
- 不允许“隐藏”字段，即给局部变量所取的名字，不可与另一个更大范围内定义的字段的名字相同（或相似）。例如，如果把一个字段叫做 **firstName**，就不要再生成一个局部变量叫做 **firstName**，或者任何易混淆的名字，如 **fistName**。
- 一行代码只声明一个变量，仅将一个变量用于一件事。

3 声明顺序

- 常量
 - 类变量
 - 公有字段
 - 受保护字段
 - 私有字段
-

语句结构

- 在设计阶段确定了软件的逻辑流结构，但构造单个语句则是编码阶段的任务。语句构造力求简单，直接，不能为了片面追求效率而使语句复杂化。

1.程序编写首先应当考虑清晰性

- 程序编写首先应当考虑清晰性，不要刻意追求技巧性，使程序编写得过于紧凑。

- 例如，有一个用 C 语句写出的程序段：

```
A[I] = A[I] + A[T];  
A[T] = A[I] - A[T];  
A[I] = A[I] - A[T];
```

- 此段程序可能不易看懂，有时还需用实际数据试验一下。
- 实际上，这段程序的功能就是交换A[I]和A[T]中的内容。目的是为了节省一个工作单元。如果改一下就能让读者一目了然了。
- ```
WORK = A[T];
A[T] = A[I];
A[I] = WORK;
```

## 2.程序要能直截了当地说明程序员的用意

■ 例如：  

```
for (int i = 1; i <= n; i++)
 for (int j = 1; j <= n; j++)
 V[i][j] = (i / j) * (j / i)
```

分析：除法运算（ / ）在除数和被除数都是整型量时，其结果只取整数部分，而得到整型量。

当  $i < j$  时， $i / j = 0$ ；当  $j < i$  时， $j / i = 0$

得到的数组 当  $i \neq j$  时

$$V[i][j] = (i / j) * (j / i) = 0$$

当  $i = j$  时

$$V[i][j] = (i / j) * (j / i) = 1$$

这样得到的结果  $V$  是一个单位矩阵。

- 写成以下的形式，就能让读者直接了解程序编写者的意图。

```
for ($i = 1; i \leq n; i++$)
 for ($j = 1; j \leq n; j++$)
 if ($i == j$)
 $V[i][j] = 1;$
 ELSE
 $V[i][j] = 0;$
```

$$\begin{pmatrix} 1 & & & 0 \\ & 1 & & \\ & & \ddots & \\ 0 & & & 1 \end{pmatrix}$$

- 
3. 除非对效率有特殊的要求，程序编写要做到清晰第一，效率第二。不要为了追求效率而丧失了清晰性。事实上，程序效率的提高主要通过选择高效的算法来实现。
  4. 首先要保证程序正确，然后才要求提高速度。反过来说，在使程序高速运行时，首先要保证它是正确的。
-

5.避免使用临时变量而使可读性下降。例如，有的程序员为了追求效率，往往喜欢把表达式

**$A[I] + 1 / A[I];$**

写成  **$B = A[I];$**

**$X = B + 1 / B;$**

这样将一句分成两句写，可读性下降。

---

6. 让编译程序做简单的优化。

7. 尽可能使用库函数

8. 避免使用空的**ELSE**语句和**IF... THEN IF...**的语句。这种结构容易使读者产生误解。例如，

```
if (char >= 'a')
```

```
if (char <= 'z')
```

```
 printf("This is a letter");
```

```
else
```

```
 printf("This is not a letter");
```

可能产生二义性问题。

---

9.避免采用过于复杂的条件测试。

10.尽量减少使用“否定”条件的条件语句。例如，如果在程序中出现

```
if (!(char < '0' || char > '9'))
```

.....

改成

```
if (char >= '0' && char <= '9')
```

.....

不要让读者绕弯子想。

- 
12. 不要修补不好的程序，要重新编写。也不要一味地追求代码的复用，要重新组织。
  13. 对太大的程序，要分块编写、测试，然后再集成。
-

---

# 第3章 软件构建

- 3.1编码概述
  - 3.2编程语言和编码规范
  - **3.3代码版本控制**
  - 3.4可以工作的类
  - 3.5高质量的子程序
  - 3.6防御式编程
  - 3.7变量和语句的构建
-

## 3.3 代码版本控制

### 问题提出

开发过程当中源代码的版本控制一直是个大问题。项目规模小了还好办，人的脑子还能记过来，项目大了，可能用各式各样的表格来记录版本信息和源代码内容，但这个办法本身的文档组织又是个问题，谁来维护？谁来更改？

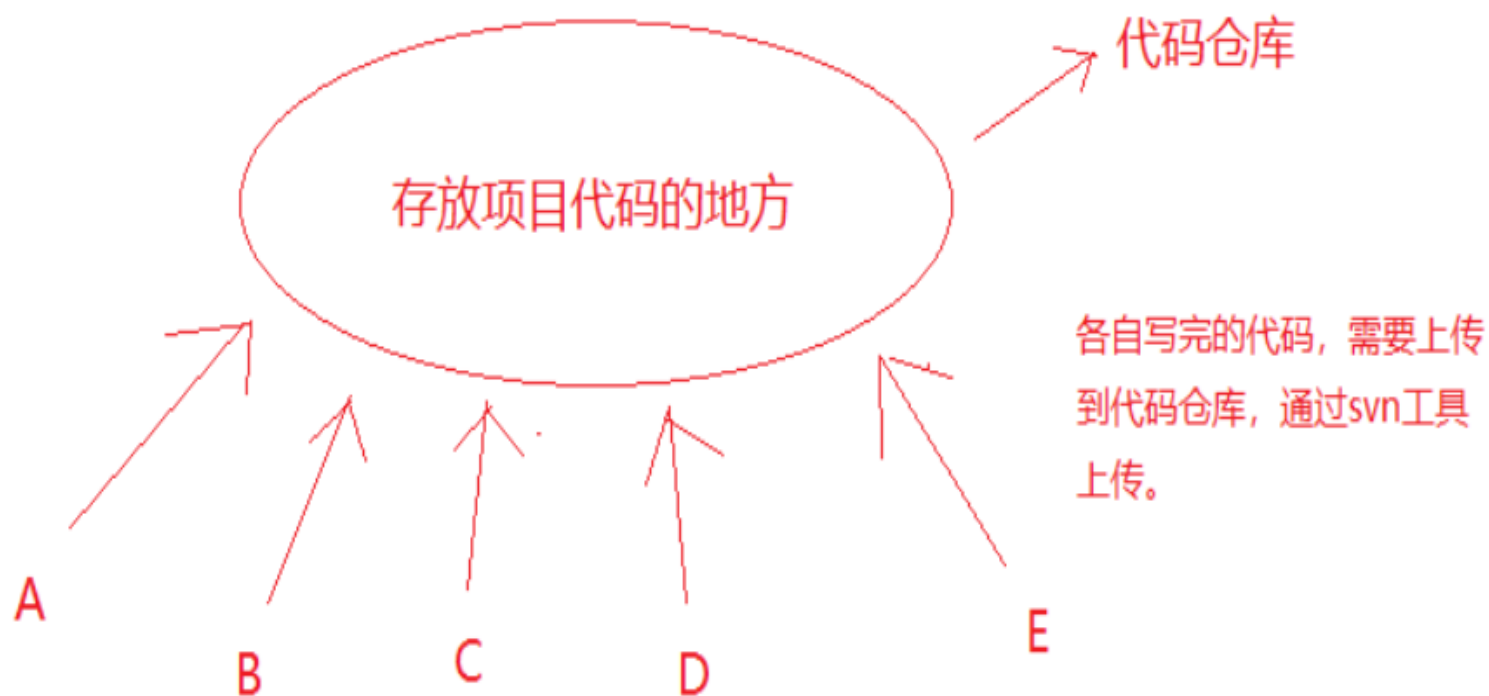
## 3.4 代码版本控制

### 问题提出

一个公司的项目有5人开发（A，B，C，D，E五个人），有一个地方专门存放项目的最新的代码，这个地方比如叫‘代码仓库’

A 写代码前需要从代码仓库下载最新的代码，然后开始写代码，需要把今天做些的代码上传到代码仓库，那这时候就要使用到SVN或GIT(版本控制工具)了。

## 问题提出



# 什么是代码版本控制

没有版本控制？

- ❖ 代码管理混乱
- ❖ 备份多个版本，占用磁盘空间大
- ❖ 解决代码冲突困难
- ❖ 容易引发BUG
- ❖ 难于追溯问题代码的修改人和修改时间
- ❖ 难于恢复至以前正确版本
- ❖ 无法进行权限控制
- ❖ 项目版本发布困难

# 什么是代码版本控制

能防止软件开发过程中意外的文件丢失、允许反追踪到早期版本、并能对版本进行分支、合并和管理。

❖ 版本控制(Revision control)是维护工程蓝图的标准做法，能追踪工程蓝图从诞生一直到定案的过程。是一种记录若干文件内容变化，以便将来查阅特定版本修订情况的系统。

# 什么是代码版本控制

在修改代码之前，要告诉版本控制工具一声。这个操作叫检出或签出，**Check Out**。

在修改完毕后，也要告诉版本控制工具一声。这个操作叫检入或签入，**Check In**。

# 主流版本控制软件

## 1. Borland Starteam

是一个集合了版本控制、构建管理（**Build Management**）和缺陷跟踪系统为一体的软件，并且具有强大的图形界面，易学易用；但管理复杂、维护困难。

## 2. PVCS Version Manager

是美国的**MERANT**公司软件配置管理工具**PVCS** 家族中的一个组成部分，它能够实现源代码、可执行文件、应用文件、图形文件和文档的版本管理；它能安全地支持软件并行开发，对多个软件版本的变更进行有效的控制管理。

## 3. ClearCase

是**IBM ROSE**构件的一部分，主要应用于复杂的产品发放、分布式团队合作、并行的开发和维护任务。可以控制word, excel, powerpoint, visio等文件格式，对于不认识的格式可以自己定义一种类型来标识。

#### 4. Visual SourceSafe (VSS)

简单易用、方便高效、与Windows操作系统及微软开发工具高度集成。

#### 5. CVS (Concurrent Versions System)

是开发源码的并发版本系统,它是面向软件开发人员的源代码版本管理解决方案。它可用于各种平台,包括 Linux、Unix Windows NT/2000/XP等等。

#### 6. SVN(SubVersion)

CVS 的主要作者 (Fogel 等等) 在他们现任公司的资助下开发了SubVersion用以替代CVS。SubVersion 的设计目的就是针对CVS 的一些弱点进行改进。SVN是一个集中式版本控制系统。

#### 7. Git

Git是一个免费和开源的分布式版本控制系统。GitHub是一个面向开源及私有软件项目的托管平台,因为只支持Git作为唯一的版本库格式进行托管,故名GitHub。

目前很多公司使用比较多的是SVN和Git.

## SVN是集中式版本控制系统，Git是分布式版本控制系统，它们的区别：

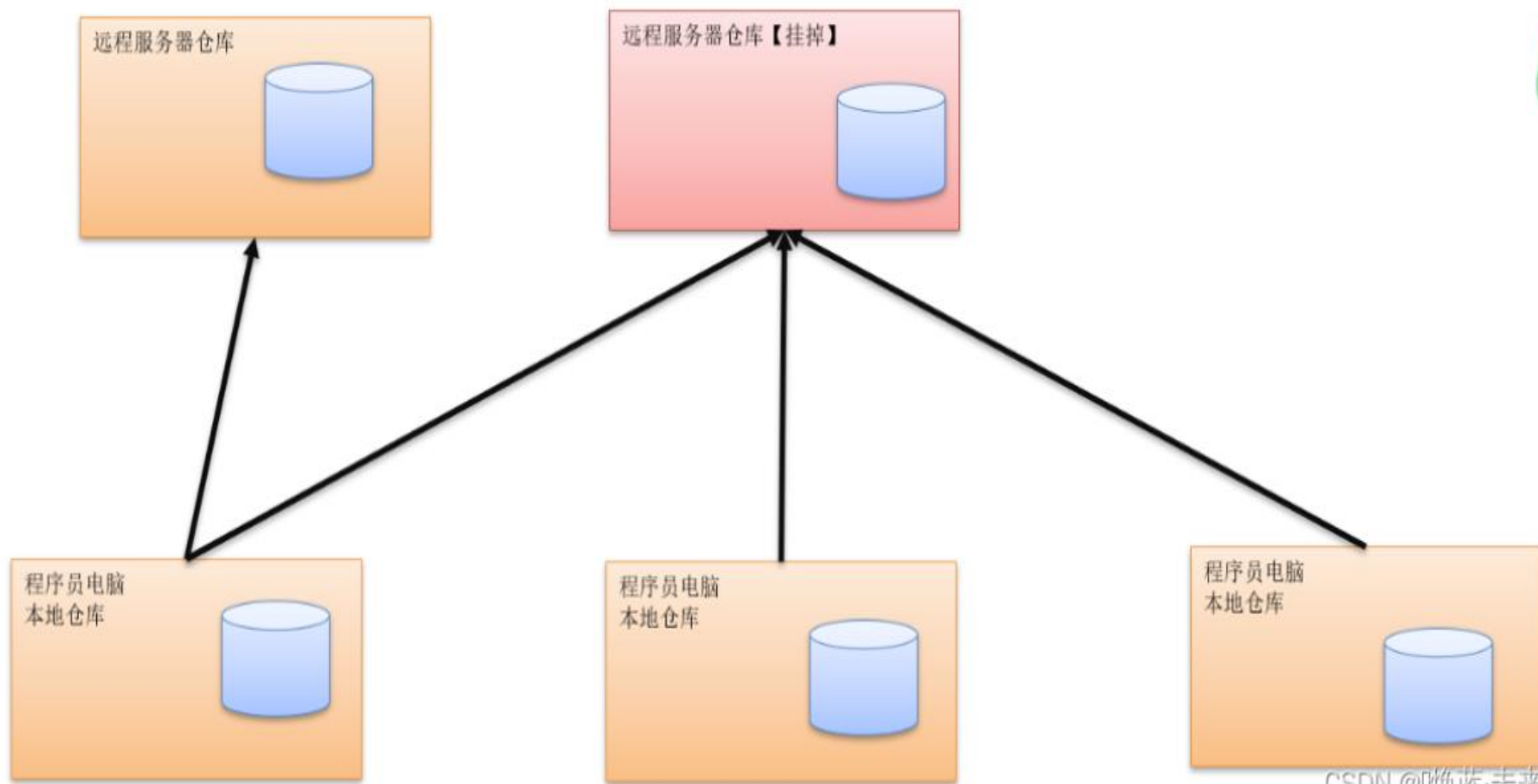
- 集中式版本控制系统就是所有人修改的都是svn服务器上的内容，而分布式版本控制系统是把服务器上的代码都clone到本地一份，开发人员修改的代码都是本地仓库的代码。
- 集中式版本控制系统需要在联网的情况下开发，而分布式版本控制系统可以在离线模式下开发。git开发人员开发完成后可以commit到本地仓库然后push到远程仓库。由于svn是直接修改服务器上的内容，如果开发人员修改到相同的内容会出现冲突。

# 版本控制好处

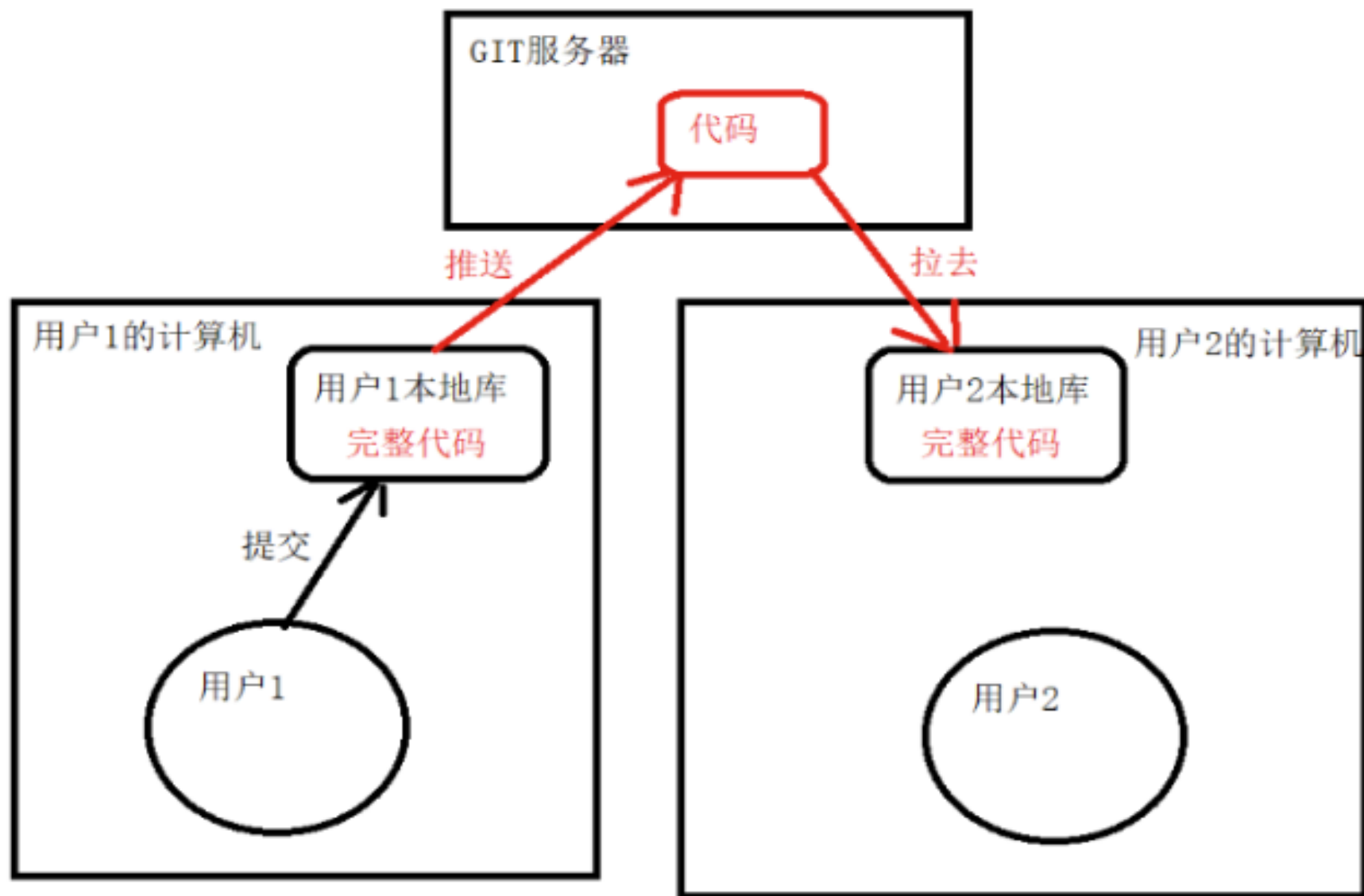
- 便于团队代码共享
- 保证整个团队使用统一的代码版本
- 能获得版本控制工具中保存的任何版本
- 能够把出错或误操作的最新版的项目恢复到正确的历史版本
- 快速的集成

# Git介绍

- Git是Linux之父Linus Torvalds花了两周时间用C写的
- Git是分布式版本控制工具



# Git介绍



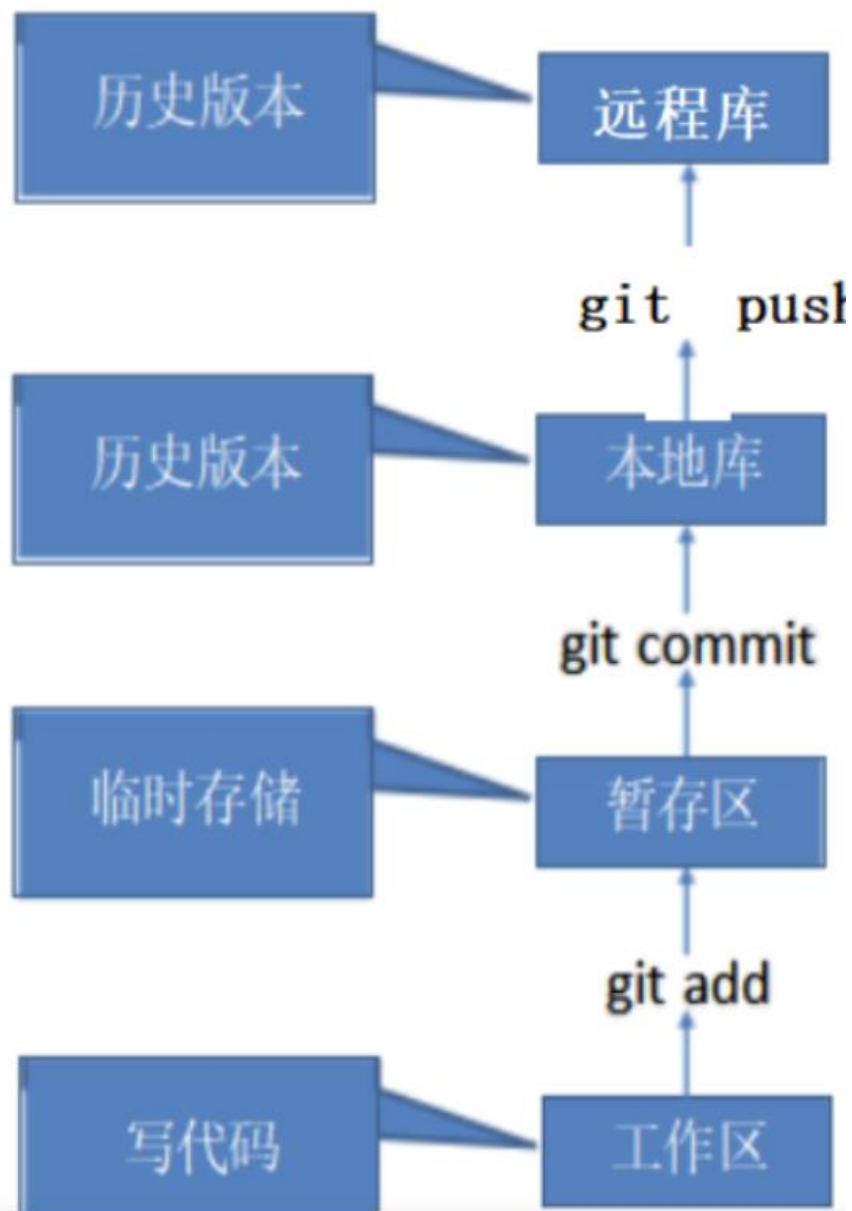
# Git介绍

Git分布式的版本控制系统解决了集中式版本控制系统的缺陷:

1. 服务器断网的情况下也可以进行开发（因为版本控制是在本地进行的）
2. 每个客户端保存的也都是整个完整的项目（包含历史记录，更加安全）

# Git 工作机制

1. **工作区**：存放代码的地方
2. **暂存区**：临时存储，将工作区的代码让git知道，通过git add将代码放到暂存区
3. **本地库**：将暂存区的代码提交到本地库，就会生成对应的历史版本，这个代码就无法删除
4. **远程库**：将本地库的代码推送到远程库

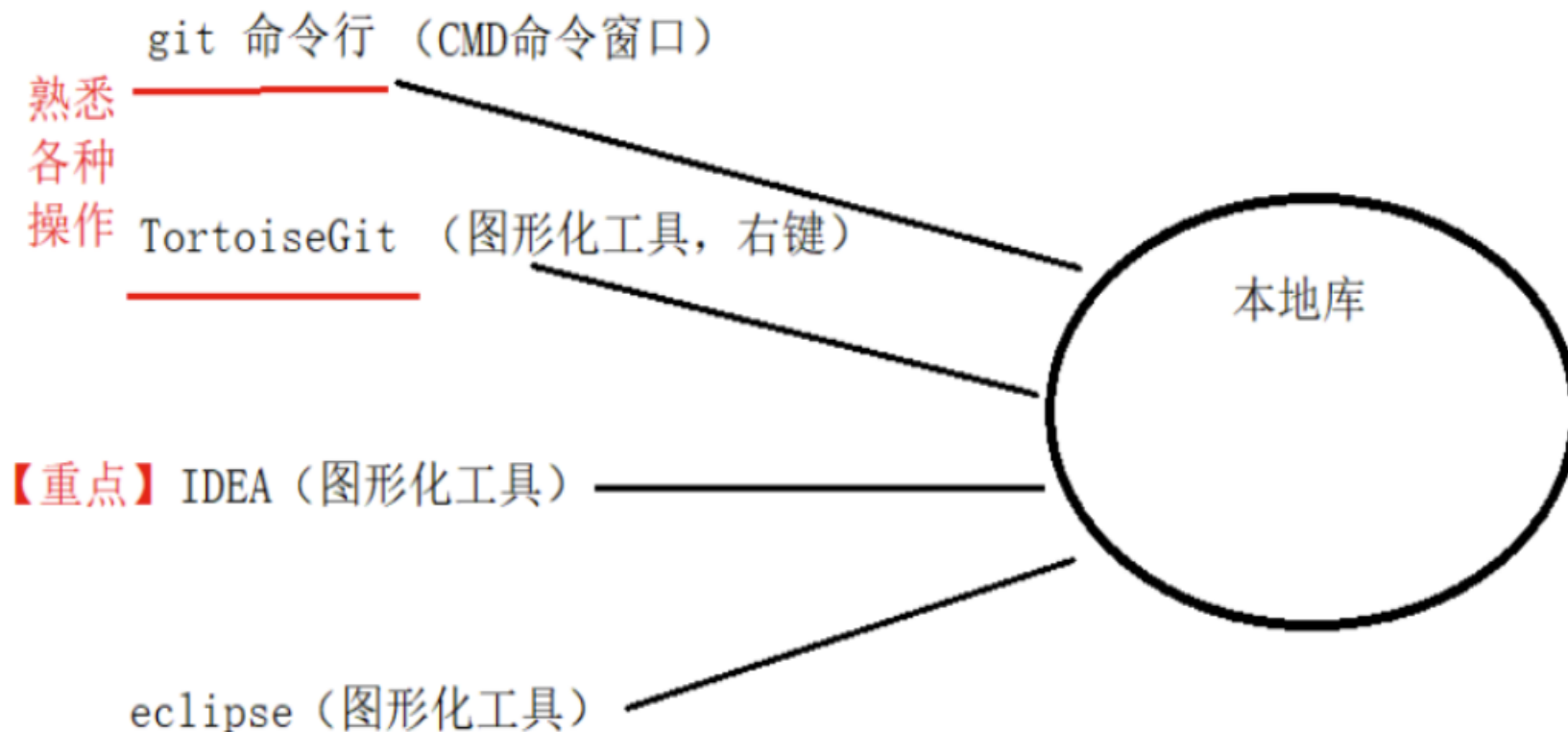


# Git 和代码托管中心

- 代码托管中心是基于网络服务器的远程代码仓库，一般我们简单称为远程库
  - 局域网
    - **GitLab**: 自己搭建 远程库
    - **gogs**: Gogs 是一款极易搭建的自助 Git 服务
  - 互联网
    - GitHub (外网)，可能无法访问
    - **Gitee 码云** (国内网站)

比较出名的代码托管中心: GitHub 和 码云

# Git 软件安装



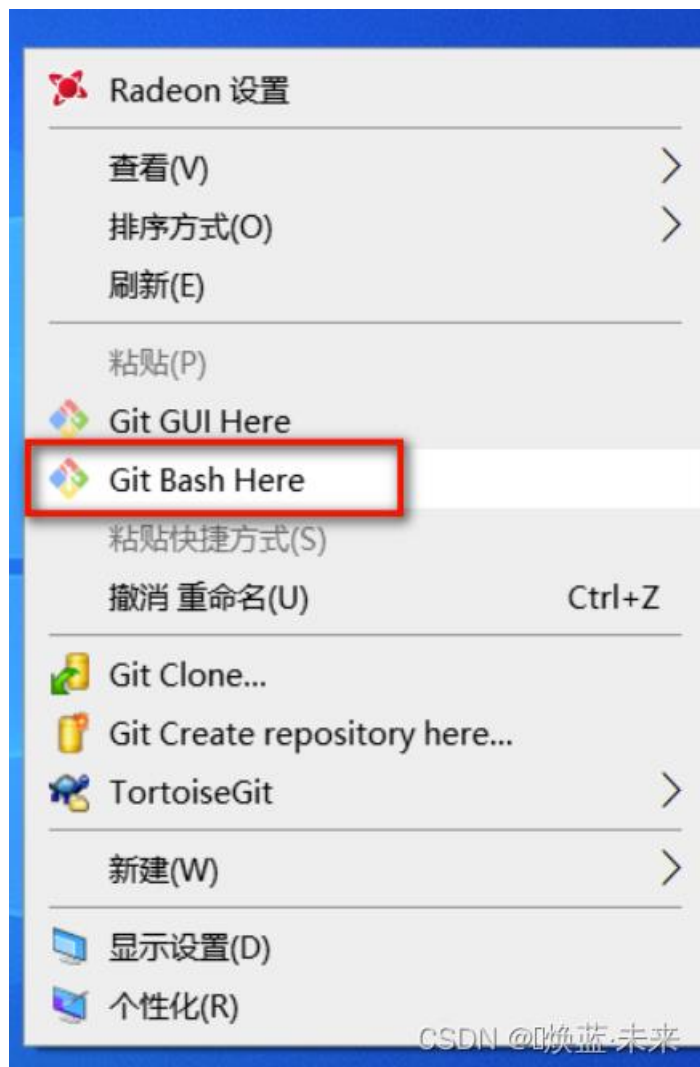
Git下载地址: <https://git-scm.com/>

TortoiseGit下载地址: <https://tortoisegit.org/download/>

# Git 常用命令

| 命令名称                              | 作用      |
|-----------------------------------|---------|
| git config --global user.name 用户名 | 设置用户名   |
| git config --global user.email 邮箱 | 设置用户邮箱  |
| git init                          | 初始化本地库  |
| git status                        | 查看本地库状态 |
| git add 文件名                       | 添加到暂存区  |
| git commit -m " 日志信息" 文件名         | 提交到本地库  |
| git reflog                        | 查看历史记录  |
| git reset --hard 版本号              | 版本穿梭    |

# Git命令实操



```
ph@PH-LAPTOP MINGW64 /d/git-space
$ git config --global user.name penghui
```

```
ph@PH-LAPTOP MINGW64 /d/git-space
$ git config --global user.email penghui@cuit.edu.cn
```

```
$ git init
Initialized empty Git repository in D:/git-space/git-test2/.git/

ph@PH-LAPTOP MINGW64 /d/git-space/git-test2 (master)
$ git status
On branch master
```

```
$ git add hello.txt
```

```
ph@PH-LAPTOP MINGW64 /d/git-space/git-test2 (master)
```

```
$ git commit -m "first commit" hello.txt
[master (root-commit) 8e7aaa0] first commit
1 file changed, 3 insertions(+)
 create mode 100644 hello.txt
```

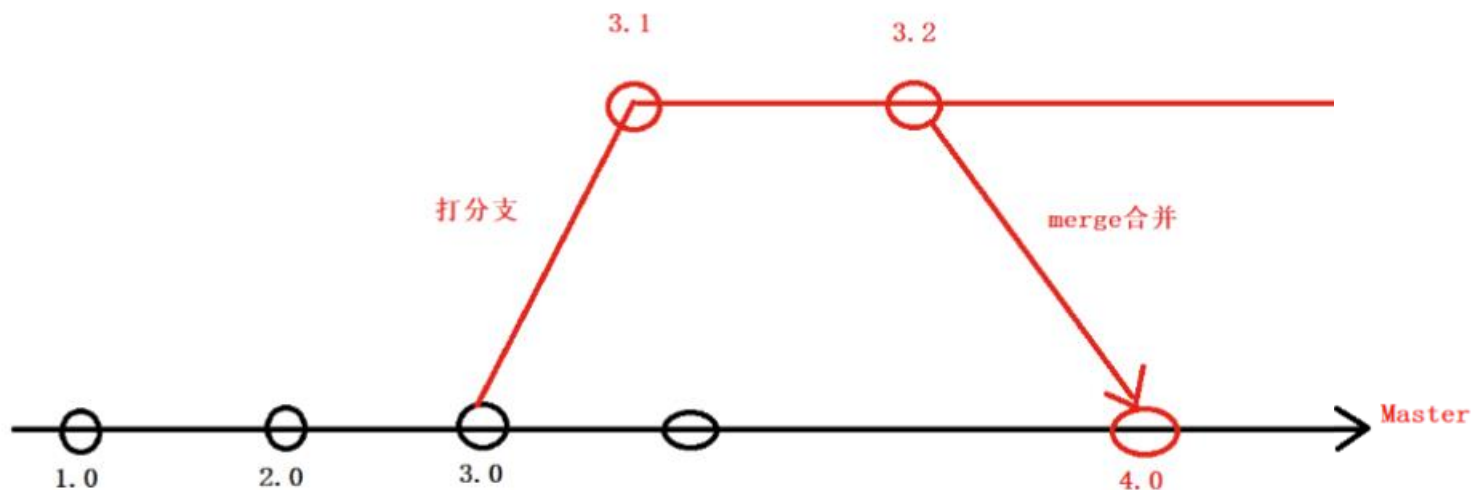
```
$ git reflog
fb30202 (HEAD -> master) HEAD@{0}: commit: second commit
8e7aaa0 HEAD@{1}: commit (initial): first commit
```

```
$ git reset --hard 8e7aaa0
HEAD is now at 8e7aaa0 first commit
```

```
$ git log
commit 8e7aaa0e473dfcfce693cc99b2c1e3d11c53db43 (HEAD -> master)
Author: penghui <penghui@cuit.edu.cn>
Date: Wed Apr 24 10:27:17 2024 +0800
```

# Git 分支操作

- 在版本控制过程中，同时推进多个任务，为每个任务，我们就可以创建每个任务的单独分支。
- 各个分支在开发过程中，如果某一个分支开发失败，不会对其他分支有任何影响。



项目正常开发，正常发布，1.0, 2.0, 3.0都正常发布了，3.0上线运行，主分支继续开发4.0，在开发4.0的过程中，发现3.0有严重bug，如何修复？修复master分支？不行，因为4.0还没有开发完成此时最好的方式就在3.0处打分支，新分支3.1，在3.1的版本中修复bug，然后3.1上线，如果3.1中还有bug，新建3.2分支，修复bug...

当4.0开发完成之后，可以将3.x中已经修复的bug merge合并到主分支

# Git 分支操作命令

| 命令名称             | 作用             |
|------------------|----------------|
| git branch 分支名   | 创建分支           |
| git branch -v    | 查看分支           |
| git checkout 分支名 | 切换分支           |
| git merge 分支名    | 把指定的分支合并到当前分支上 |

# Git 分支命令实操

```
ph@PH-LAPTOP MINGW64 /d/git-space/git-test2 (master)
$ git branch -v
* master 8e7aaa0 first commit

ph@PH-LAPTOP MINGW64 /d/git-space/git-test2 (master)
$ git branch dev

ph@PH-LAPTOP MINGW64 /d/git-space/git-test2 (master)
$ git branch -v
 dev 8e7aaa0 first commit
* master 8e7aaa0 first commit

ph@PH-LAPTOP MINGW64 /d/git-space/git-test2 (master)
$ git checkout dev
Switched to branch 'dev'
```

```
ph@PH-LAPTOP MINGW64 /d/git-space/git-test2 (master)
$ git checkout dev
Switched to branch 'dev'

ph@PH-LAPTOP MINGW64 /d/git-space/git-test2 (dev)
$ git add hello.txt

ph@PH-LAPTOP MINGW64 /d/git-space/git-test2 (dev)
$ git commit -m "3rd dev commit" hello.txt
[dev e4bb99b] 3rd dev commit
1 file changed, 1 insertion(+), 1 deletion(-)

ph@PH-LAPTOP MINGW64 /d/git-space/git-test2 (dev)
$ git checkout master
Switched to branch 'master'

ph@PH-LAPTOP MINGW64 /d/git-space/git-test2 (master)
$ get merge dev
bash: get: command not found
```

# Git 分支命令实操

冲突产生的表现：后面状态为 **MERGING**

冲突产生的原因：

合并分支时，两个分支在同一个文件的同一个位置有两套完全不同的修改。Git 无法替我们决定使用哪一个。必须人为决定新代码内容

```
hello.txt - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<<<<<<< HEAD
hello,阿萨 llo 王45
=====
hello,hello, 阿萨德 2333 dhello,
>>>>>>> dev
hello 34人 world
hello world,hello world,hello world
adfasdfa
```

```
MINGW64:/d/git-space/git-test2
ph@PH-LAPTOP MINGW64 /d/git-space/git-test2 (master|MERGING)
$ git merge dev
error: Merging is not possible because you have unmerged files.
hint: Fix them up in the work tree, and then use 'git add/rm <file>'
hint: as appropriate to mark resolution and make a commit.
fatal: Exiting because of an unresolved conflict.

ph@PH-LAPTOP MINGW64 /d/git-space/git-test2 (master|MERGING)
$ git add hello.txt

ph@PH-LAPTOP MINGW64 /d/git-space/git-test2 (master|MERGING)
$ git commit -m "4 commit" hello.txt
fatal: cannot do a partial commit during a merge.

ph@PH-LAPTOP MINGW64 /d/git-space/git-test2 (master|MERGING)
$ git commit -m "4 commit"
[master 95ca528] 4 commit

ph@PH-LAPTOP MINGW64 /d/git-space/git-test2 (master)
$ git merge dev
Already up to date.

ph@PH-LAPTOP MINGW64 /d/git-space/git-test2 (master)
$
```

# Gitee码云操作

码云网址: <https://gitee.com/>

创建远程仓库



仓库名称 \* ✓

git0820

归属

yuanxinqi

路径 \* ✓

git0820

仓库地址: <https://gitee.com/yuanxinqi/git0820>

仓库介绍

用简短的语言来描述一下吧

☐ 开源 (所有人可见)

☒ 私有 (仅仓库成员可见)

☐ 企业内部开源 (仅企业成员可见) ?

☐ 初始化仓库 (设置语言、.gitignore、开源许可证)

☐ 设置模板 (添加 Readme、Issue、Pull Request 模板文件)

☒ 选择分支模型 (仓库创建后将根据所选模型创建分支)

单分支模型 (只创建 master 分支)

创建

# Git远程仓库操作命令

| 命令名称                   | 作用                           |
|------------------------|------------------------------|
| git remote -v          | 查看当前所有远程地址别名                 |
| git remote add 别名 远程地址 | 起别名（第一次）                     |
| git push 别名 分支         | 推送本地分支上的内容到远程仓库              |
| git clone 远程地址         | 将远程仓库的内容克隆到本地（第一次）           |
| git pull 远程库地址别名 远程分支名 | 将远程仓库对于分支最新内容拉下来后与当前本地分支直接合并 |

# Git远程仓库命令实操

HTTPS SSH SVN SVN+SSH

<https://gitee.com/phui33/git0424.git>

## 克隆远程仓库到本地

```
ph@PH-LAPTOP MINGW64 /d/git-space/git-clone2
$ git clone https://gitee.com/phui33/git0424.git
Cloning into 'git0424'...
remote: Enumerating objects: 19, done.
remote: Counting objects: 100% (19/19), done.
remote: Compressing objects: 100% (13/13), done.
remote: Total 19 (delta 3), reused 0 (delta 0), pack-reused
Receiving objects: 100% (19/19), done.
Resolving deltas: 100% (3/3), done.
```

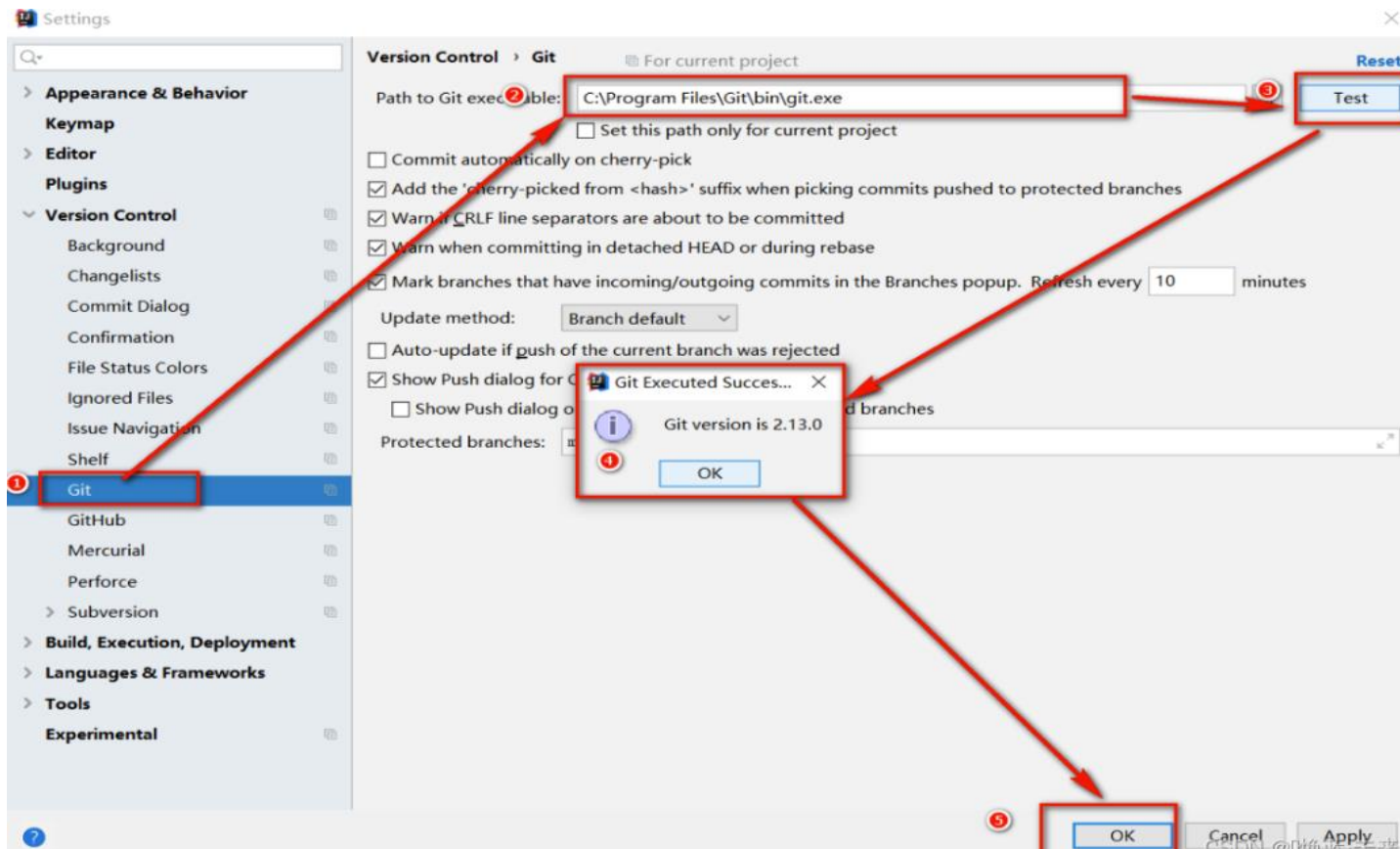
```
ph@PH-LAPTOP MINGW64 /d/git-space/git-test2 (master)
$ git checkout master
Already on 'master'

ph@PH-LAPTOP MINGW64 /d/git-space/git-test2 (master)
$ git push git0424 master
Enumerating objects: 16, done.
Counting objects: 100% (16/16), done.
Delta compression using up to 16 threads
Compressing objects: 100% (11/11), done.
Writing objects: 100% (16/16), 1.32 KiB | 135.00 KiB/s, done.
Total 16 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Powered by GITEE.COM [GNK-6.4]
To https://gitee.com/phui33/git0424.git
 * [new branch] master -> master
```



## 选择邀请合作者

# IDEA集合Git



Git学习网站:

[https://blog.csdn.net/weixin\\_47824895/article/details/130169142](https://blog.csdn.net/weixin_47824895/article/details/130169142)

这章第二部分见 《第三章 软件构建-2 》 PPT